

1 Problem Analysis: Prominence Calculation

1.1 Problem Statement

Given a sequence of N altitude measurements, we need to calculate the *prominence* of each mountain peak. The prominence of a peak is defined as the height difference between the peak and the lowest valley (lowest point of the connecting ridge) that separates it from any higher peak.

1.2 Key Observations

Observation 1: Symmetric Definition

The prominence of a peak can be computed as the minimum of two values:

- The lowest valley to the left of the peak, before encountering a higher peak
- The lowest valley to the right of the peak, before encountering a higher peak

Observation 2: Stack-Based Approach

When processing peaks from left to right, a new peak of height h eliminates all previous peaks with height $\leq h$ from consideration. This is because any valley between them and the new peak becomes irrelevant for prominence calculation. We can use a stack to maintain peaks in **strictly decreasing order** of height.

Observation 3: Mirroring Trick

After computing left-side prominence, we can mirror the entire array and compute right-side prominence using the same algorithm, then take the minimum.

1.3 Algorithm

1. Read input and remove consecutive duplicates (they don't affect peak/-valley classification)
2. If $C = 1$, simply count the number of local maxima and output
3. Otherwise, for each direction ($d = 0$ for left-to-right, $d = 1$ for right-to-left):
 - Maintain a stack of peak indices with strictly decreasing heights
 - For each peak at position i , store $\text{prom}[d][i] = a[i]$
 - While top of stack has height $\leq a[i]$, pop the stack (updating the deepest valley)
 - If stack is non-empty, prominence to this direction is $a[i] - \text{valley}[\text{top}]$
 - Push current peak index and update deepest valley for this peak
4. After mirroring, combine results: for each peak, $\text{prominence} = \min(\text{prom}[0][i], \text{prom}[1][N - 1 - i])$
5. Output all positive prominence values

1.4 Correctness Proof

We prove that the algorithm outputs the correct prominence for each peak.

Lemma 1: Stack Invariant

After processing positions 0 through i , the stack contains exactly those peaks among positions $0 \dots i$ that have strictly decreasing heights and are higher than all peaks between them and position i .

Proof: Initially the stack is empty, satisfying the invariant. When we encounter a new peak at position i with height h :

- We pop all peaks with height $\leq h$. Each popped peak is either lower than h or equal to h . Since h is a peak, any valley between the popped peak and position i is visible from h .
- We push the new peak. Since we popped all lower/equal peaks, the remaining stack peaks are all higher than h , maintaining strict decreasing order.

The invariant is preserved. $\square$

Lemma 2: Left Prominence Calculation

For a peak at position i , $\text{prom}[0][i]$ computed in the left-to-right pass equals the height of the lowest valley that separates this peak from any higher peak on its left.

Proof: Consider the stack after processing position i . By Lemma 1, the top of stack (if exists) is the closest higher peak to the left of i . The variable $\text{valley}[\text{top}]$ stores the deepest valley encountered between that higher peak and position i (updated when we pop smaller peaks). Therefore, $a[i] - \text{valley}[\text{top}]$ is exactly the prominence defined by the saddle point to the left. $\square$

Lemma 3: Right Prominence Calculation

After mirroring and performing the same algorithm, $\text{prom}[1][N - 1 - i]$ equals the lowest valley to the right of peak i before any higher peak.

Proof: After mirroring, position $N - 1 - i$ in the mirrored array corresponds to position i in the original array. The right side in the original becomes the left side in the mirrored array. Applying Lemma 2 to this pass gives the claim. $\square$

Theorem: Final Answer

For each peak i , the algorithm outputs $\min(\text{prom}[0][i], \text{prom}[1][N - 1 - i])$, which equals the true prominence of peak i .

Proof: By definition, prominence is the minimum of the two ridge heights connecting the peak to higher terrain on either side. Lemma 2 shows $\text{prom}[0][i]$ is the left ridge height, and Lemma 3 shows $\text{prom}[1][N - 1 - i]$ is the right ridge height. The minimum of these two values is exactly the prominence. $\square$

1.5 Complexity Analysis

- **Time Complexity:** Each element is pushed to the stack exactly once and popped at most once. The two passes give $O(N)$ time.
- **Space Complexity:** The arrays peak, valley, and prom each store at most N elements, yielding $O(N)$ memory.

1.6 Reference Implementation

```
#include <stdio.h>
#define NMax 1000005
long N, C, s;
long a[NMax];

long min_(long i, long j) { if (i < j) return i; return j; }
long peak[NMax], valley[NMax], top = -1;
long prom[2][NMax];

void popTop() {
    if (top < 0) { return; }
    if (top >= 1 && valley[top-1] > valley[top]) {
        valley[top-1] = valley[top];
    }
    top--;
}

int main() {
    freopen("proeminenta.in", "r", stdin);
    freopen("proeminenta.out", "w", stdout);

    scanf("%ld%ld", &C, &N);
    for (int i = 0; i < N; i++) {
        scanf("%ld", &a[i]);
        if (i > 0 && a[i] == a[i-1]) {
            i--; N--; // eliminate consecutive duplicates
        }
    }
    if (C == 1) {
        for (int i = 1; i < N-1; i++) {
            if (a[i-1] < a[i] && a[i] > a[i+1]) { s++; }
        }
        printf("%ld\n", s);
        return 0;
    }

    for (int d = 0; d <= 1; d++) {
        for (int i = 1; i < N-1; i++) {
            if (a[i-1] < a[i] && a[i] > a[i+1]) { // peak found

```

```

        prom[d][i] = a[i];
        while (top >= 0 && a[peak[top]] <= a[i]) {
            popTop();
        }
        if (top >= 0 && a[peak[top]] > a[i]) {
            prom[d][i] = a[i] - valley[top];
        }
        peak[++top] = i;
        valley[top] = a[i];
    }
    if (top >= 0 && valley[top] > a[i]) {
        valley[top] = a[i];
    }
}
// Mirror the array for the reverse pass
for (int i = 0; i < N / 2; i++) {
    long aux = a[i]; a[i] = a[N-1-i]; a[N-1-i] = aux;
}
top = -1;
}
for (int i = 1; i < N-1; i++) if (a[i-1] < a[i] && a[i]
> a[i+1]) {
    if (prom[0][i] > prom[1][N-1-i]) {
        prom[0][i] = prom[1][N-1-i];
    }
}

for (int i = 1; i < N-1; i++) {
    if (prom[0][i] > 0) { printf("%ld_", prom[0][i]); }
}

printf("\n");
fclose(stdout);
return 0;
}

```

1.7 Implementation Notes

- The `popTop()` function also propagates the deepest valley when removing a peak from the stack, ensuring that valley information is not lost.
- Consecutive duplicate altitudes are removed at input time since they don't create valid peaks or valleys.
- The final combination step takes the minimum of left and right prominences, as required by the definition.
- Only peaks with positive prominence are output, as requested by the problem specification.